МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. Шухова»**
**(БГТУ им. В. Г. Шухова)**

ИИТУС
БГТУ им. В.Г. Шухова

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И УПРАВЛЯЮЩИХ СИСТЕМ

Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

**Лабораторная работа №2**

по дисциплине: «Спецификация, арх. и ППС»

по теме: «Анализ требований, выбор архитектуры программного обеспечения и организация командной разработки»

Выполнил: студент группы ПВ-233

Алексеенко Андрей Евгеньевич

Проверил: преподаватель

Доденко Олег Павлович

Белгород, 2026 г.

**Цель работы:** получить практические навыки анализа требований к программному обеспечению, проектирования архитектуры информационной системы, формирования команды разработки и распределения задач между участниками проекта.

**Содержание**


Задание 1 ... 3

1. Название проекта ... 3
2. Назначение системы ... 3
3. Целевая аудитория ... 3
4. Основные проблемы, которые решает система ... 3
5. Основные функции системы ... 3
6. Предполагаемые условия эксплуатации ... 4

Задание 2 ... 4

1. Функциональные требования ... 4
2. Нефункциональные требования ... 5

Задание 3 ... 6

1. Монолитная архитектура ... 6
2. Микросервисная архитектура ... 6
3. Многослойная архитектура ... 7

Задание 4 ... 7

1. Введение ... 7
2. Основания для разработки ... 7
3. Назначение и цели создания системы ... 8
4. Требования к системе ... 8
5. Состав и содержание работ ... 9
6. Порядок контроля и приемки ... 9
7. Требования к составу и содержанию документации ... 9
8. Источники разработки ... 9

Задание 5 ... 9

Задание 6 ... 11

Вывод ... 12

**Задание 1.** Описание проекта.

**1.** Название проекта

Информационная система «FoodExpress» (Сервис доставки еды).

**2.** Назначение системы

Система представляет собой комплексную платформу (B2B2C), предназначенную для автоматизации полного цикла заказа и доставки готовой еды. Она выступает в роли цифрового посредника, объединяющего локальные рестораны, службу доставки и конечных потребителей. Система обеспечивает прозрачный процесс выбора блюд, онлайн-оплаты и логистики, минимизируя ручное вмешательство и сокращая время выполнения заказа.

**3.** Целевая аудитория

Целевая аудитория делится на четыре основные группы. Клиенты (B2C) — физические лица, заказывающие еду домой, в офис или на мероприятия; для них важны удобный интерфейс, скорость, надежность и возможность отслеживания заказа. Рестораны-партнеры (B2B) — заведения общественного питания, желающие расширить каналы сбыта без затрат на собственную доставку; им необходим инструмент для управления меню, ценами и стоп-листами в реальном времени. Курьеры — сотрудники службы доставки (собственной или партнерской), нуждающиеся в мобильном приложении с навигацией, четкими маршрутами и статусами заказов. Администраторы и менеджеры сервиса — сотрудники компании FoodExpress, отвечающие за модерацию ресторанов, разрешение спорных ситуаций, техподдержку и аналитику.

**4.** Основные проблемы, которые решает система

Система решает несколько ключевых проблем. Во-первых, это фрагментация рынка: клиентам приходится искать номера телефонов разных ресторанов и изучать их меню в соцсетях, что отнимает много времени; FoodExpress агрегирует все предложения в одном окне. Во-вторых, сложность логистики: оформление заказа по телефону часто приводит к ошибкам в позициях, адресах и времени; система автоматизирует передачу заказа на кухню и курьеру. В-третьих, отсутствие прозрачности: позвонив в ресторан, клиент не знает, когда еда будет готовиться и когда выедет курьер; система предоставляет трекинг со статусами «Готовится», «В пути» и картой. В-четвертых, потеря выручки в часы пик: без автоматизации рестораны не справляются с потоком звонков, тогда как система принимает неограниченное количество заказов одновременно.

**5.** Основные функции системы

К основным функциям системы относятся: модуль аутентификации (регистрация и вход по номеру телефона с подтверждением через SMS); каталог и поиск (просмотр списка ресторанов, фильтрация по кухням, рейтингу и времени

доставки, текстовый поиск по блюдам); корзина и оформление (добавление блюд, выбор адреса доставки с интеграцией с картами, выбор времени, применение промокодов); онлайн-оплата (интеграция с платежным шлюзом для оплаты банковскими картами, СБП или Apple/Google Pay); личный кабинет ресторана (управление меню, добавление блюд, скрытие позиций в стоп-лист, просмотр входящих заказов); модуль доставки для курьеров (получение заказа, навигация до ресторана и клиента, изменение статуса заказа); панель администратора (управление пользователями, аналитика заказов, обработка возвратов и жалоб).

**6.** Предполагаемые условия эксплуатации

Условия эксплуатации различаются в зависимости от роли пользователя. Клиентская часть включает мобильные приложения для iOS и Android (основной канал), а также адаптивную веб-версию (PWA) для работы через браузеры Chrome, Safari и Firefox. Серверная часть размещается в отказоустойчивой облачной инфраструктуре (например, Yandex Cloud или AWS) с балансировкой нагрузки для обеспечения бесперебойной работы 24/7. Оборудование ресторанов включает планшеты (iOS/Android) или стационарные ПК (Windows) с установленным веб-клиентом для приема заказов. Оборудование курьеров включает смартфоны с GPS-модулем и стабильным мобильным интернетом (3G/4G/5G) для работы навигации и трекинга.

**Задание 2.** Формирование требований.

**1.** Функциональные требования

| ID | Название | Описание | Пользователь | Ожидаемый результат |
|---|---|---|---|---|
| F-01 | Авторизация | Вход по номеру телефона и SMS-коду | Клиент | Доступ к личному кабинету |
| F-02 | Корзина | Добавление блюд в корзину, изменение количества | Клиент | Формирование заказа |
| F-03 | Оплата | Оплата картой онлайн | Клиент | Статус заказа «Оплачен» |
| F-04 | Управление меню | Добавление/редактирование блюд | Ресторан | Обновление меню на сайте |
| F-05 | Трекинг | Отображение статуса доставки на карте | Клиент | Знание, где курьер |

Представленные функциональные требования охватывают ключевые пользовательские сценарии, необходимые для работы сервиса доставки. Они обеспечивают базовую авторизацию, процесс формирования и оплаты заказа, а также инструменты для партнеров (управление меню) и клиентов (отслеживание доставки). Данный набор требований является минимально необходимым для запуска первой

версии продукта (MVP) и в дальнейшем может быть расширен функциями промокодов, рейтингов и чата с поддержкой.

**2.** Нефункциональные требования

Требования к надежности. Система должна обеспечивать высокий уровень доступности, не менее 99.9% времени в год (около 8.76 часов простоя в год). Должна быть предусмотрена отказоустойчивость: выход из строя одного сервера не должен приводить к полной остановке сервиса. Все критически важные данные (заказы, платежи) должны сохраняться в базах данных с ежедневным резервным копированием и возможностью восстановления на любую точку во времени (Point-in-Time Recovery).

Требования к производительности. Время загрузки страницы меню ресторана не должно превышать 1 секунды при скорости соединения не ниже 10 Мбит/с. Обработка запроса на создание заказа и возврат статуса «Оплачен» должны занимать не более 2 секунд. Система должна выдерживать пиковые нагрузки в вечернее время (с 18:00 до 22:00) и в пятницу, обрабатывая не менее 500 одновременных запросов в секунду без деградации скорости отклика.

Требования к безопасности. Все данные, передаваемые между клиентом и сервером, должны быть зашифрованы с использованием протоколов SSL/TLS (версии не ниже 1.2). Система должна соответствовать требованиям Федерального закона № 152-ФЗ «О персональных данных» в части хранения и обработки персональных данных клиентов (шифрование данных в состоянии покоя, разграничение прав доступа). Аутентификация должна осуществляться по одноразовым паролям (SMS), а платежные данные должны обрабатываться через сертифицированный платежный шлюз (без хранения номеров карт на серверах компании).

Требования к масштабируемости. Архитектура системы должна поддерживать горизонтальное масштабирование, позволяя добавлять новые вычислительные узлы (серверы) без остановки работы сервиса. При росте количества пользователей и ресторанов-партнеров система должна сохранять заявленную производительность за счет распределения нагрузки (балансировки) и кэширования часто запрашиваемых данных (меню ресторанов). Масштабирование должно быть автоматическим при достижении заданных порогов нагрузки (например, загрузка CPU выше 70%).

Требования к условиям эксплуатации. Клиентская часть системы должна корректно работать во всех современных веб-браузерах (Chrome, Safari, Firefox, Edge) последних версий, а также на мобильных устройствах под управлением iOS (версии 14 и выше) и Android (версии 9 и выше). Серверная часть должна быть развернута в облачной инфраструктуре с возможностью мониторинга (сбор логов, метрик) и оповещения администраторов о сбоях в режиме реального времени (24/7). Интерфейс системы должен поддерживать два языка (русский и английский) для возможности масштабирования на международный рынок в будущем.

**Задание 3.** Проектирование архитектуры.

**1.** Монолитная архитектура

Монолитная архитектура предполагает проектирование системы как единого, неделимого приложения. Основными компонентами в данном подходе выступают клиентская часть (Frontend), серверная часть (Backend) и единая база данных. Принцип взаимодействия компонентов строится следующим образом: пользователь через браузер или мобильное приложение отправляет запрос на сервер. Сервер обрабатывает запрос в рамках единого процесса: выполняет бизнес-логику (проверка данных, расчет стоимости), обращается к базе данных для чтения или записи информации и возвращает готовый ответ клиенту.

Хранение данных осуществляется в единой реляционной базе данных (например, PostgreSQL), которая содержит таблицы для всех сущностей: пользователи, рестораны, меню, заказы и платежи. Развертывание системы производится в виде одного Docker-контейнера или на одном виртуальном сервере, что упрощает процесс установки.

К преимуществам данного подхода относятся быстрота разработки на начальном этапе, простота тестирования и легкость развертывания. К недостаткам можно отнести сложность масштабирования: если нагрузка возрастает на модуль заказов, приходится масштабировать всё приложение целиком, что неэффективно. Кроме того, любая ошибка в коде может привести к падению всего сервиса.

**2.** Микросервисная архитектура

В микросервисной архитектуре система разбивается на независимые, слабосвязанные сервисы, каждый из которых отвечает за свою бизнес-функцию и может разрабатываться и развертываться отдельно. В рамках проекта FoodExpress выделяются следующие сервисы.

Сервис «Пользователи» отвечает за регистрацию, авторизацию, управление профилем и историей заказов клиента. Он хранит данные в собственной базе данных Users и взаимодействует с другими сервисами через REST API. Сервис «Каталог» отвечает за хранение и выдачу информации о ресторанах, их меню, ценах и стоп-листах. Он имеет собственную базу данных Restaurants/Menu и предоставляет данные для клиентского приложения. Сервис «Заказы» является центральным элементом, отвечающим за корзину, оформление заказа, расчет стоимости и отслеживание статуса. Он хранит данные в базе Orders и взаимодействует с сервисом «Каталог» (для получения цен на блюда) и сервисом «Платежи». Сервис «Платежи» отвечает за интеграцию с банковским шлюзом, проведение транзакций и возвратов, храня данные о транзакциях в базе Transactions.

Взаимодействие между сервисами осуществляется через легковесные протоколы, такие как REST API или брокер сообщений (например, RabbitMQ) для

асинхронной передачи событий. Для управления маршрутизацией запросов от клиентов используется API Gateway, который принимает все внешние запросы и перенаправляет их в нужный сервис.

**3.** Многослойная архитектура

Многослойная архитектура предполагает логическое разделение системы на горизонтальные уровни, где каждый слой выполняет строго определенную роль и взаимодействует только с соседними слоями.

Слой представления отвечает за взаимодействие с пользователем. Его основными компонентами являются веб-интерфейс и мобильные приложения. Этот слой не содержит бизнес-логики, он лишь отображает данные, полученные от нижнего слоя, и передает действия пользователя на слой бизнес-логики. Слой бизнес-логики является ядром системы. Он содержит сервисы обработки заказов, расчета скидок, валидации данных и управления статусами. На этом уровне реализуются все правила предметной области (например, проверка того, что заказ не может быть создан, если ресторан закрыт). Слой доступа к данным отвечает за взаимодействие с базой данных. Он скрывает от бизнес-логики детали работы с СУБД. Компонентами этого слоя являются ORM-библиотеки (например, SQLAlchemy), которые преобразуют объекты программы в SQL-запросы и обратно, а также обеспечивают управление транзакциями.

Взаимодействие между слоями происходит сверху вниз: запрос от пользователя поступает на слой представления, передается на слой бизнес-логики, который обращается к слою доступа к данным для получения или сохранения информации в базе данных.

**Задание 4.** Составление технического задания.

**1.** Введение

Настоящее техническое задание определяет требования к разработке информационной системы «FoodExpress», представляющей собой сервис доставки еды. Система предназначена для автоматизации процессов заказа, оплаты и логистики готовых блюд от ресторанов-партнеров до конечных потребителей. Краткое описание системы включает в себя клиентское мобильное приложение и веб-интерфейс, серверную часть с базой данных, а также панели управления для ресторанов и курьеров.

**2.** Основания для разработки

Основанием для выполнения проекта является учебное задание по дисциплине «Спецификация, архитектура и ППС». Требования заказчика (в лице преподавателя) заключаются в необходимости продемонстрировать навыки анализа предметной

области, проектирования архитектуры и формирования технической документации для заданного программного продукта.

**3.** Назначение и цели создания системы

Система «FoodExpress» предназначена для предоставления пользователям удобного способа заказа еды с возможностью онлайн-оплаты и отслеживания доставки. Целями создания системы являются: автоматизация процесса приема заказов, снижение нагрузки на рестораны за счет цифрового канала продаж, обеспечение прозрачной логистики доставки, а также повышение удовлетворенности клиентов за счет быстрого и качественного сервиса.

**4.** Требования к системе

Функциональные требования. Система должна обеспечивать выполнение следующих функций: авторизацию пользователей по номеру телефона с подтверждением через SMS-код; просмотр каталога ресторанов и их меню; добавление блюд в корзину и изменение их количества; оформление заказа с выбором адреса и времени доставки; онлайн-оплату банковской картой; отображение текущего статуса заказа и местоположения курьера на карте; управление меню и стоп-листами для ресторанов-партнеров; администрирование заказов и пользователей через панель управления.

Требования к надежности. Система должна обеспечивать доступность не менее 99.9% времени в год. Время восстановления после сбоя не должно превышать 30 минут. Все критически важные данные (заказы, транзакции) должны сохраняться в базах данных с ежедневным резервным копированием и возможностью восстановления.

Требования к производительности. Время загрузки страницы меню ресторана не должно превышать 1 секунды. Обработка запроса на создание и оплату заказа должна занимать не более 2 секунд. Система должна выдерживать пиковые нагрузки (500 одновременных запросов в секунду) в вечернее время и в выходные дни без деградации скорости отклика.

Требования к безопасности. Передача данных между клиентом и сервером должна осуществляться с использованием протоколов SSL/TLS. Система должна соответствовать требованиям Федерального закона № 152-ФЗ «О персональных данных» в части хранения и обработки персональных данных. Аутентификация должна быть двухфакторной (SMS). Платежные данные должны обрабатываться через сертифицированный платежный шлюз без хранения номеров карт на серверах компании.

Требования к условиям эксплуатации. Клиентская часть системы должна корректно работать в браузерах Chrome, Safari, Firefox, Edge, а также на мобильных устройствах под управлением iOS и Android. Серверная часть должна быть

развернута в облачной инфраструктуре с возможностью мониторинга (сбор логов, метрик) и оповещения администраторов о сбоях в режиме реального времени.

**5.** Состав и содержание работ

Разработка системы включает следующие этапы. Анализ предметной области и формирование требований (2 недели), в результате чего создается настоящее ТЗ. Проектирование архитектуры и дизайна интерфейса (2 недели), результатом является технический проект и макеты. Разработка программного обеспечения (8 недель), включающая написание серверной и клиентской части, а также настройку баз данных. Тестирование и отладка (2 недели), результатом которых становится отчет о тестировании и устранение выявленных дефектов.

**6.** Порядок контроля и приемки

Контроль и приемка системы осуществляются путем демонстрации работы программного продукта. Критериями приемки являются: успешное прохождение всех сценариев тестирования (создание заказа, оплата, трекинг), отсутствие критических ошибок, а также соответствие функционала требованиям, изложенным в разделе 4 настоящего ТЗ.

**7.** Требования к составу и содержанию документации

В процессе разработки должна быть подготовлена следующая документация: техническое задание (настоящий документ), руководство пользователя (для клиентов, ресторанов и курьеров), а также документация API (Swagger/OpenAPI) для интеграции с внешними системами.

**8.** Источники разработки

Источниками разработки являются учебные материалы по дисциплине «Спецификация, архитектура и ППС», открытые стандарты проектирования, а также техническая документация используемых фреймворков и библиотек.

**Задание 5.** Формирование команды разработки.

Для реализации проекта «FoodExpress» была сформирована команда из 9 человек. Состав команды соответствует выбранной микросервисной архитектуре и разделению зон ответственности между сервисами.

**1.** Руководитель проекта

Основные обязанности: планирование сроков и этапов разработки, управление бюджетом и ресурсами, коммуникация с заказчиком, разрешение конфликтов внутри команды, контроль рисков.

Задачи: составление плана проекта, проведение ежедневных встреч (Daily Scrum), контроль выполнения задач, отчетность перед заказчиком.

Результат работы: утвержденный план проекта, своевременная сдача этапов, итоговый отчет о выполнении проекта.

**2.** Системный аналитик

Основные обязанности: анализ предметной области, выявление требований, проектирование структуры данных, написание технической документации, постановка задач разработчикам.

Задачи: написание технического задания (ТЗ), создание ER-диаграмм базы данных, описание пользовательских сценариев (Use Cases), составление схем API.

Результат работы: утвержденное ТЗ, схемы баз данных, техническая документация, понятные задачи для разработчиков.

**3.** Backend-разработчик

Основные обязанности: разработка серверной логики, создание API для взаимодействия с клиентской частью и другими сервисами, интеграция с базой данных.

Задачи: написание кода на Python (FastAPI), реализация логики корзины и оформления заказа, настройка ORM (SQLAlchemy), оптимизация запросов к БД.

Результат работы: работающий код серверной части, документация API (Swagger), стабильные эндпоинты для оформления заказов.

**4.** Backend-разработчик

Основные обязанности: разработка модулей аутентификации и интеграции с платежным шлюзом, обеспечение безопасности данных.

Задачи: реализация регистрации и входа по SMS, интеграция с банковским API, настройка шифрования и защиты персональных данных.

Результат работы: безопасный модуль авторизации, успешно работающий функционал оплаты, защищенные каналы передачи данных.

**5.** Frontend-разработчик

Основные обязанности: разработка пользовательского интерфейса, обеспечение удобства и адаптивности, интеграция с API бэкенда.

Задачи: верстка страниц (каталог, корзина, личный кабинет), реализация интерактивных элементов (фильтры, карта), подключение к API через AJAX/Fetch.

Результат работы: готовое веб-приложение, интуитивно понятный интерфейс для клиентов, корректное отображение данных.

**6.** Frontend-разработчик

Основные обязанности: разработка мобильного приложения для платформ iOS и Android, реализация навигации и GPS-трекинга.

Задачи: создание интерфейса для получения заказов, интеграция с картографическими сервисами, реализация фонового отслеживания местоположения.

Результат работы: мобильное приложение с навигацией, которое позволяет курьерам оперативно получать заказы и строить маршруты.

**7.** Разработчик базы данных (DBA)

Основные обязанности: проектирование и настройка базы данных, обеспечение целостности данных, оптимизация производительности, резервное копирование.

Задачи: создание структуры таблиц (PostgreSQL), настройка индексов, написание сложных SQL-запросов, настройка репликации и бэкапов.

Результат работы: спроектированная и настроенная БД, высокопроизводительная схема данных, регулярные резервные копии.

**8.** Тестировщик

Основные обязанности: обеспечение качества продукта, выявление дефектов, проверка соответствия функционала требованиям.

Задачи: написание тест-кейсов, ручное тестирование (функциональное, UI), автоматизация тестов (Unit, API), создание баг-репортов.

Результат работы: отчеты о тестировании, список найденных и исправленных дефектов, уверенность в стабильности релиза.

**9.** DevOps-инженер

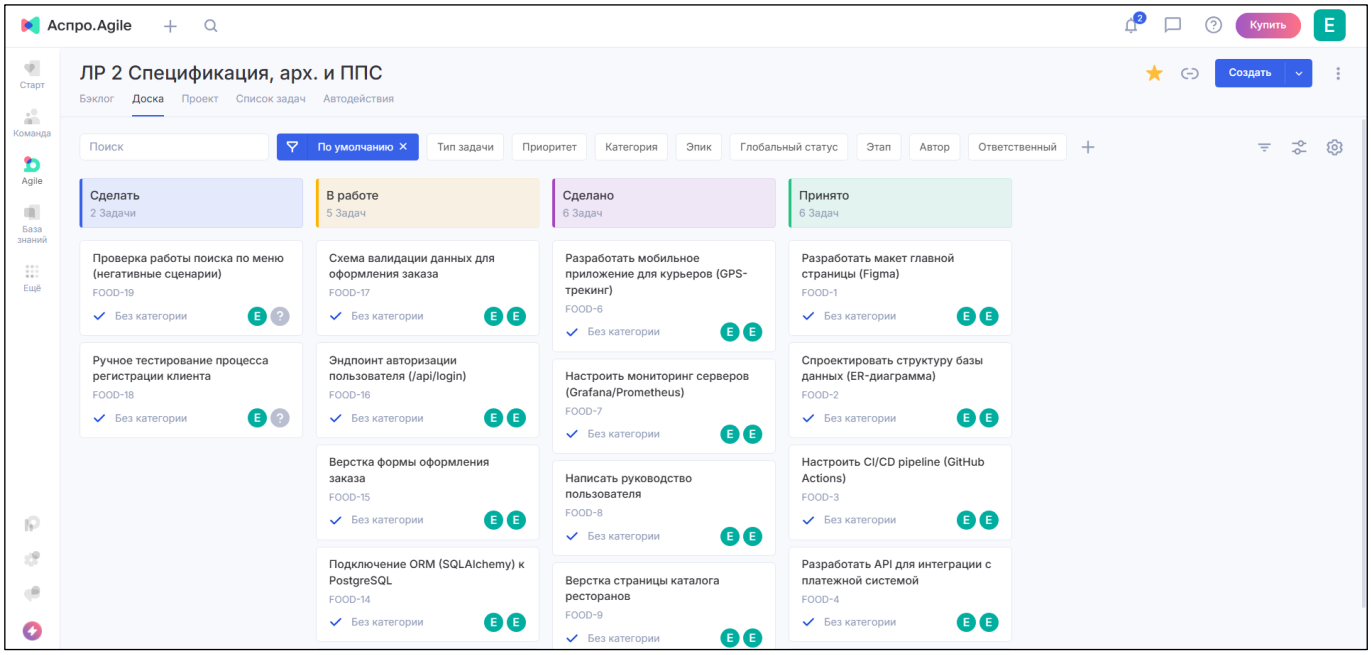
Основные обязанности: настройка инфраструктуры, автоматизация сборки и развертывания, мониторинг работы системы.

Задачи: настройка CI/CD (GitHub Actions), контейнеризация приложений (Docker), развертывание в облаке (Yandex Cloud), настройка мониторинга (Grafana/Prometheus).

Результат работы: работающая инфраструктура, автоматическое развертывание обновлений, стабильная работа серверов 24/7.

**Задание 6.** Организация доски задач.

Для управления задачами проекта «FoodExpress» и визуализации рабочего процесса была использована методология Kanban. Доска создана в системе Аспро.Agile.




Представленная Kanban-доска позволяет наглядно отслеживать прогресс разработки, ограничивать количество задач в работе (WIP-limit) и оперативно выявлять «узкие места» в процессе (например, скопление задач на этапе проверки). Использование префикса FOOD и четкая декомпозиция задач (до уровня конкретных эндпоинтов и макетов) делают процесс разработки прозрачным и управляемым.

**Вывод:** выполняя лабораторную работу, я освоил полный цикл проектирования информационной системы: от анализа требований и составления ТЗ по ГОСТ до выбора архитектуры и организации командной разработки. На практике я убедился, что успех проекта зависит не только от кода, но и от правильной декомпозиции задач и распределения ролей. Использование Kanban-доски и сравнение трех архитектурных подходов позволило мне понять, как строить работу так, чтобы система была надежной, масштабируемой и удобной для пользователей.